

การประยุกต์ใช้โครงข่ายประสาทเทียมที่ใช้ข้อมูลเชิงฟิสิกส์เพื่อประเมินราคาออปชัน

(Application of Physics-Informed Neural Networks for Option Pricing)

ผู้จัดทำ: นายพรชัย คำแดง

Email: pornchaikhamdaeng@gmail.com Github: github.com/pornchai1889

วันที่: 20 มกราคม 2569

บทคัดย่อ

การวิจัยนี้มีวัตถุประสงค์เพื่อศึกษาและพัฒนาแบบจำลองโครงข่ายประสาทเทียมที่ใช้ข้อมูลเชิงฟิสิกส์ (Physics-Informed Neural Networks; PINN) เพื่อหาผลเฉลยสมการเชิงอนุพันธ์ย่อยแบบลิค-โบลล์ (Black-Scholes Partial Differential Equation; BSPDE) สำหรับการประมาณราคาออปชันสไตล์ยุโรป (European Options Pricing) ทั้งแบบ Call และ Put (ในงานวิจัยนี้เลือกบิตคอยน์เป็นสินทรัพย์อ้างอิง) รวมถึงค่า Greeks โมเดลที่พัฒนาถูกออกแบบให้เรียนรู้พลวัตของราคาตามสมการ BSPDE โดยตรง โดยไม่ต้องอาศัยข้อมูลราคาตลาดในการสอน (Unsupervised Learning via Physics Laws) โดยใช้รูปแบบการสุ่มแบบกำหนดได้ (การวิจัยนี้เน้นไปที่การสุ่มแบบ Uniform Distribution) ผลการทดลองแสดงให้เห็นว่าโมเดล PINN สามารถทำนายราคาออปชันได้สอดคล้องกับผลเฉลยเชิงวิเคราะห์ (Analytical Solution) ในหลายสัญญาณ ซึ่งจะช่วยเพิ่มประสิทธิภาพในการบริหารจัดการพอร์ตการลงทุนแบบตามเวลาจริง (Real-time) ได้อย่างมีนัยสำคัญ

1. บทนำ

ในตลาดอนุพันธ์ที่มีการซื้อขายสัญญาที่มูลค่าแปรผันตามสินทรัพย์อ้างอิง เคยมีความท้าทายสำคัญคือเราไม่ทราบอย่างแน่ชัดว่าราคายุติธรรม (Fair Value) หรือค่าพรีเมียมมันคือเท่าไรและการมาของแบบจำลองการเงินของแบล็ก-โบลล์ ที่มีพื้นฐานมาจากฟิสิกส์ของการเคลื่อนที่แบบบราวน์ ผสมกับการแก้สมการเชิงแคลคูลัสสุ่ม (Stochastic calculus) และการใช้ตรรกะทางการเงินแบบไม่เก็งกำไร (No-Arbitrage) จึงนำไปสู่การค้นพบสมการ BSPDE ในที่สุด ซึ่งสร้างประโยชน์อย่างมหาศาลต่อการประเมินราคาออปชันในเวลาต่อมา ในช่วงไม่กี่ปีที่ผ่านมาเทคโนโลยี Deep learning ซึ่งเป็นหัวใจสำคัญของปัญญาประดิษฐ์ (AI) ได้พัฒนาไปไกลและถูกประยุกต์ใช้อย่างหลากหลาย รวมถึงนวัตกรรมโครงข่ายประสาทเทียมที่ใช้ข้อมูลเชิงฟิสิกส์ (Physics-Informed Neural Networks; PINN) ที่ได้ผสมผสานศักยภาพของโครงข่ายประสาทเทียมเข้ากับกฎทางฟิสิกส์ (สมการอนุพันธ์) เพื่อยกระดับความแม่นยำในการคำนวณให้แก่เรา

งานวิจัยนี้ค้นพบว่า แบบจำลองแบล็ก-โบลล์สามารถนำมาใช้เป็น กฎทางฟิสิกส์เพื่อกำกับกระบวนการเรียนรู้ของ AI ได้อย่างมีประสิทธิภาพ และสามารถประมาณค่าผลเฉลยของสมการ PDE ได้สอดคล้องกับผลเฉลยเชิงวิเคราะห์อย่างยิ่ง (จากการทดสอบบนสัญญาณจริงพบว่าค่า RMSE = 88.89 และ Corr = 0.9999) นอกจากนี้งานวิจัยยังได้ออกแบบกระบวนการเรียนรู้ให้ครอบคลุมปริภูมิพารามิเตอร์ที่หลากหลายของสมการ BSPDE ได้แก่ ราคาสินทรัพย์ (S) เวลา (t) ราคาใช้สิทธิ (K) อัตราผลตอบแทนปราศจากความเสี่ยง (r) และความผันผวน (σ) ผลคือโมเดลได้มองเห็นภาพรวมของเหตุการณ์และความเป็นไปได้เกือบทั้งหมดของตลาดอนุพันธ์

2. ทฤษฎี

แบบจำลองแบล็ก-โชลส์ ซึ่งนำเสนอโดย ฟิชเชอร์ แบล็ก (Fischer Black) และ ไมรอน โชลส์ (Myron Scholes) ในปี 1973 และพัฒนาต่อโดย โรเบิร์ต เมอร์ตัน (Robert Merton) ถือเป็นแบบจำลองทางคณิตศาสตร์ที่เป็นรากฐานและปฏิวัติวงการวิศวกรรมการเงินแบบจำลองนี้ให้วิธีการเชิงปริมาณในการประเมินราคาอนุพันธ์ทางการเงินประเภท European Option โดยสร้างความเชื่อมโยงที่ลึกซึ้งระหว่างทฤษฎีความน่าจะเป็นทางการเงินและสมการเชิงอนุพันธ์ย่อยในทางฟิสิกส์ จุดเริ่มต้นของแบบจำลองแบล็ก-โชลส์ ตั้งอยู่บนสมมติฐานหลักที่ว่า ราคาของสินทรัพย์อ้างอิง (S) มีการเคลื่อนที่ ตามกระบวนการ Geometric Brownian Motion (GBM) ซึ่งเป็นสมการเชิงอนุพันธ์สุ่ม (Stochastic Differential Equation; SDE)

$$dS_t = \mu S_t dt + \sigma S_t dW_t \quad (1)$$

ผลลัพธ์ของการรวม SDE, Itô's Lemma และตรรกะ No-Arbitrage เข้าด้วยกันอย่างสมบูรณ์ และการจัดองค์ประกอบของพจน์การสุ่ม (dW_t) ออกไปได้ ผลลัพธ์ที่ได้คือสมการเชิงอนุพันธ์ย่อย

$$\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0 \quad (2)$$

อย่างไรก็ตามเวลา (t) จากสมการ (2) ยังเป็นกระแสเวลาแบบไปข้างหน้าเพื่อให้ PINN โมเดลได้เรียนรู้สถานะตลาดอุปชันแบบภาพรวม งานวิจัยนี้จึงพิจารณาที่ระยะเวลาคงเหลือจนถึงวันครบกำหนดอายุ (τ) หรือ Time to Maturity แทนจึงได้สมการว่า

$$\mathcal{F}(V) := \frac{\partial V}{\partial \tau} - \left(\frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV \right) = 0 \quad (3)$$

เมื่อ $\mathcal{F}(V)$ คือฟังก์ชันความผิดเพี้ยนจากกฎฟิสิกส์ (PDE Residual)

โดยงานวิจัยนี้จะประมาณค่าคำตอบของสมการ $V(S, \tau)$ โดยใช้โครงข่ายประสาทเทียมเชิงลึก (Deep Neural Network) $\hat{V}(S, \tau, K, r, \sigma; \theta)$ ซึ่งพารามิเตอร์ θ (Weights และ Biases) ของโครงข่ายจะถูกปรับค่าให้เหมาะสมที่สุด (Optimize) ด้วยการลดฟังก์ชันความสูญเสียรวม $\mathcal{L}_{total}$ ผ่านการใช้กฎทางฟิสิกส์และเงื่อนไขขอบเขตอย่างเคร่งครัดดังนี้

$$\mathcal{L}_{total} = \lambda_{PDE} \mathcal{L}_{PDE} + \lambda_{IVP} \mathcal{L}_{IVP} + \lambda_{BVP} \mathcal{L}_{BVP} + \lambda_{Kink} \mathcal{L}_{Kink} \quad (4)$$

องค์ประกอบของฟังก์ชันความสูญเสียในแต่ละส่วนถูกนิยามในรูปแบบของค่าความคลาดเคลื่อนกำลังสองเฉลี่ย (MSE) ดังนี้

$$\mathcal{L}_{PDE} = \frac{1}{N_{PDE}} \sum_{i=1}^{N_{PDE}} \left(\frac{\partial \hat{V}}{\partial \tau} - \left[\frac{1}{2} \sigma^2 S^2 \frac{\partial^2 \hat{V}}{\partial S^2} + rS \frac{\partial \hat{V}}{\partial S} - r\hat{V} \right] \right)^2 \quad (5)$$

$$\mathcal{L}_{IVP} = \frac{1}{N_{IVP}} \sum_{i=1}^{N_{IVP}} \left(\hat{V}(S_i, 0) - \text{Payoff}(S_i) \right)^2 \quad (6)$$

$$\mathcal{L}_{BVP} = \frac{1}{N_{BVP}} \left[\sum_{i=1}^{N_{BVP}} \left(\hat{V}(S_{min}, \tau_i) - V_{LB} \right)^2 + \sum_{i=1}^{N_{BVP}} \left(\hat{V}(S_{max}, \tau_i) - V_{UB} \right)^2 \right] \quad (7)$$

$$\mathcal{L}_{Kink} = \frac{1}{N_{Kink}} \sum_{i=1}^{N_{Kink}} \left(\hat{V}(K_i, 0) \right)^2 \quad (8)$$

สมการ (5) คือฟังก์ชันความสูญเสียของสมการ BSPDE นั้นเอง หรือ $\mathcal{F}(V)$ ในสมการ (3) เพื่อบังคับให้โมเดลต้องทำนายค่าอปชันให้สอดคล้องกับสมการแบล็ก-โพลล์ สมการที่ (6) คือฟังก์ชันความสูญเสียของค่าขอบเริ่มต้น (Initial value points; IVP) เพื่อบังคับให้โมเดลต้องทำนายค่าอปชันที่เวลา 0 ออกมาเท่ากับมูลค่าการใช้สิทธิ ณ วันครบกำหนดอายุ ($\text{Payoff}(S_i)$) สมการ (7) คือฟังก์ชันความสูญเสียของค่าขอบ (Boundary value points; BVP) เพื่อบังคับให้โมเดลต้องทำนายราคาอปชันที่สินทรัพย์อ้างอิงที่มีมูลค่าต่ำสุดและสูงสุด (S_{min} และ S_{max}) ให้สอดคล้องกับค่าขอบจริง (V_{LB} และ V_{UB}) และสมการที่ (6) คือฟังก์ชันความสูญเสียของค่าจุดหักมุมของราคา $S = K$ และ $\tau = 0$ ซึ่งเป็นจุดๆที่ราคาอปชันจะมีค่าเป็น 0 ทั้ง call และ put โดยโมเดลจะต้องเรียนรู้จุดหักมุมนี้ เพื่อให้เห็นภาพของเงื่อนไขค่าขอบยิ่งขึ้นว่าโมเดลได้เรียนรู้อะไรสามารถเขียนออกมาได้ดังนี้

Initial Condition (Payoff at $\tau = 0$):

$$V(S, 0) = \max(S - K, 0) \quad \text{for Call Option}$$

$$V(S, 0) = \max(K - S, 0) \quad \text{for Put Option}$$

Lower Boundary ($S \rightarrow 0$):

$$V(S, \tau) = 0 \quad \text{for Call Option}$$

$$V(S, \tau) = Ke^{-r\tau} \quad \text{for Put Option}$$

Upper Boundary ($S \rightarrow S_{max}$):

$$V(S, \tau) = Ke^{-r\tau} \quad \text{for Call Option}$$

$$V(S, \tau) = 0 \quad \text{for Put Option}$$

โมเดลจะเรียนรู้การทำนายราคาอปชันทั้งหมดผ่านการใช้กฎทางฟิสิกส์และเงื่อนไขขอบเขตอย่างเคร่งครัด เพื่อให้เห็นว่าโมเดลทำนายได้สอดคล้องกับแบบจำลองแบล็ก-โพลล์ จึงต้องเทียบผลลัพธ์กับผลเฉลยเชิงวิเคราะห์ (Analytical Solution) ที่มีสมการดังนี้

$$V_{call} = S \cdot \Phi(d_1) - Ke^{-r\tau} \cdot \Phi(d_2) \quad (9)$$

$$V_{put} = Ke^{-r\tau} \cdot \Phi(-d_2) - S \cdot \Phi(-d_1) \quad (10)$$

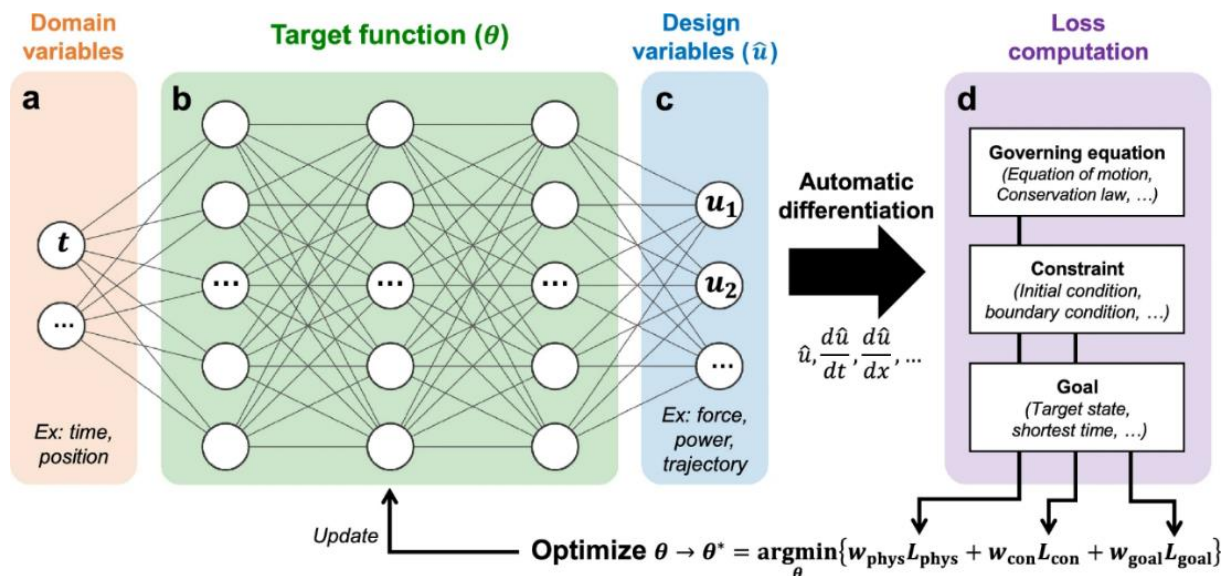
เมื่อ

$$d_1 = \frac{\ln(S/K) + \left(r + \frac{\sigma^2}{2}\right)\tau}{\sigma\sqrt{\tau}}, \quad d_2 = d_1 - \sigma\sqrt{\tau} \quad (11)$$

$$\Phi(d) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^d e^{-\frac{1}{2}x^2} dx \quad (12)$$

สถาปัตยกรรมที่ให้เอไอได้เรียนรู้กฎฟิสิกส์การเงินคือ PINNs ที่บุกเบิกและพัฒนาอย่างเป็นระบบโดย Maziar Raissi และคณะ ซึ่งพิสูจน์ให้เห็นในงานวิจัยในปี 2019 แล้วว่าเครือข่ายประสาทเทียมเชิงลึก (Deep Neural Network; DNN) เมื่อถูกฝึกให้เรียนรู้สมการฟิสิกส์จะสามารถเข้าใจกายภาพของระบบธรรมชาติได้ เช่น สามารถทำนายพื้นผิวผลเฉลยของสมการ PDE แบบไม่เป็นเชิงเส้นได้แม้จะมีข้อมูลจริงจำกัดหรือไม่มีผลเฉลยจริงเลยก็ตาม ตัวอย่างเช่น การใช้ PINNs ในการหาผลเฉลยของ Nonlinear Schrödinger Equation (NLSE) สำหรับอธิบายการแพร่ของคลื่นแสงในเส้นใยนำแสง (Optical fibers) อีกทั้ง PINNs ยังสามารถใช้ค้นหาพารามิเตอร์ที่ไม่ทราบค่าของระบบได้ด้วยหรือการเรียนรู้แบบย้อนกลับ (Inverse problem) เช่นการค้นหาค่าคงที่ๆไม่ทราบค่าจากสมการฟิสิกส์ของไหล (Navier-Stokes) โดย

ใช้ข้อมูลจริงบางส่วนจากการทดลองเป็นต้น ต่อมาปลายปี 2023 Ashish Dhiman และ Yibei Hu ก็ได้ต่อยอดแนวคิดของ PINNs มาใช้กับระบบการเงินกับสมการที่โด่งดังอย่าง Black-Scholes Partial Differential Equation แล้วค้นพบว่า PINNs สามารถเข้าใจกายภาพของตลาดการเงินอนุพันธ์ได้เป็นอย่างดี



ภาพ 1 โครงข่ายประสาทเทียมที่ใช้ข้อมูลเชิงฟิสิกส์ (PINN) มีกระบวนการทำงานเริ่มต้นจากการนำตัวแปรนำเข้า (a) ผ่านการประมวลผลด้วยโครงข่ายประสาทเทียม (b) เพื่อทำนายตัวแปรเป้าหมายที่ออกแบบไว้ (c) จากนั้นจึงทำการปรับค่าพารามิเตอร์ให้เหมาะสมที่สุดผ่านฟังก์ชันความสูญเสียรวม (d) ซึ่งมีการกำกับด้วยข้อกำหนดทางฟิสิกส์

ที่มา. จาก *nature*, โดย Jaemin Seo, สืบค้นเมื่อ 21 มกราคม 2569, จาก <https://www.nature.com/articles/s41598-023-49977-3>

3. วิธีดำเนินการวิจัย

จากทฤษฎีที่ได้ศึกษามาเราสามารถออกแบบโมเดลให้เรียนรู้สภาพตลาดการเงินอนุพันธ์ได้อย่างครอบคลุมที่สุดทั้ง Call และ Put รวมถึงได้ทุกประเภทสัญญา เพื่อทดสอบว่าเมื่อโมเดลได้เรียนรู้ข้อมูลแบบหลายมิติ (S, τ, K, r, σ) ด้วยสถาปัตยกรรม PINN จะยังสามารถทำนายราคาอปชันออกมาได้สอดคล้องกับผลเฉลยเชิงวิเคราะห์ด้วยการตั้งค่าดังนี้

ตาราง 1

ตัวแปรข้อมูลการฝึก PINN Model

หมวดหมู่	ตัวแปร	ค่า/ช่วง
Market Domain	Spot Price, S	[0, 1000000]
	Strike Price, K	116000
	Strike Step, K_{step}	1000
	Time to Maturity, τ	[0.0, 0.26]
	Volatility, σ	[0.1, 2.0]
	Risk Free rate, r	[0.0, 0.15]
	Types of Options	Call or Put
Model Architecture	Network Structure	4 Layers $\times$ 256 Neurons
	Activation Functions	Hidden: Tanh / Output: Softplus
	Input Dimension	5 (S, τ, K, r, σ)
	Output Dimension	1 (Option Price, V)
Sampling Strategy	Moneyness Range (S/K)	[0.0, 2.0]
	Batch Sampling	10000
	PDE Multiplier	4.0
	Kink Multiplier	0.5
Training Config	Total Epochs	100000
	Learning Rate	1×10^{-4} (Adam Optimizer)
Loss Weights	λ_{PDE}	1.0
	λ_{IVP}	20.0
	λ_{BVP}	20.0
	λ_{Kink}	20.0

ที่มา. จากไฟล์การทดลอง configs/train_config.yaml

3.1 กระบวนการทำงาน

โครงสร้างระบบถูกแบ่งออกเป็นโมดูลาร์ (Modular) เพื่อรองรับการขยายตัวที่ดีขึ้น (Scalability) ความปลอดภัย และการพัฒนาต่อยอดในอนาคต เมื่อเสร็จสิ้นการตั้งค่าตามตาราง 1 (ในไฟล์ `configs/train_config.yaml`) ให้ใช้คำสั่ง `python train.py --config configs/train_config.yaml` หรือกดรันที่ไฟล์ `train.py` จากนั้นระบบจะทำงานตามขั้นตอนดังต่อไปนี้

3.1.1 ระบบจะโหลดค่า config เข้าสู่หน่วยความจำและอ่านค่าประเภทของอปชันว่าเป็น call หรือ put (*physics_engine*) เพื่อเลือกประเภทของฟิสิกส์ให้ตรงกับลอจิกของอปชันนั้นๆ นำค่า config เข้าสู่กระบวนการเทรน (*trainer.train()*) ผ่าน *class Trainer* (`src/core/trainer.py`)

3.1.2 สร้างข้อมูลสมนอร์มัลไลซ์ IVP (S, τ, K, r, σ) และ V (*payoff*) ตามเงื่อนไข *Initial Condition* ผ่าน *class DataGenerator* เป็นจำนวน n ชุด (n_sample_data) บนช่วงตามตาราง 1 โดยพิจารณาที่ $\tau = 0$ อย่างเดียว โดยค่า K จะถูกสุ่มแบบไม่ต่อเนื่อง ($K_step: 1000.0$) ขณะที่ S จะมีการสุ่มแบบ Uniform รอบค่า K โดยมีค่าต่ำสุดคือ 0 และค่าสูงสุดคือสองเท่าของ K (อิงตาม Moneyness Range) ส่วน r และ σ ก็จะถูกสุ่มตามช่วงของตัวเองตามตาราง 1 หลังจากนั้นให้โมเดลทำนายค่า V โดยใช้ข้อมูลสมนอร์มัลไลซ์ IVP (S, τ, K, r, σ) พร้อมกับคำนวณหาค่า $\mathcal{L}_{IVP}$ ผ่าน *nn.MSELoss()* เทียบกับ V (*payoff*)

3.1.3 สร้างข้อมูลสมนอร์มัลไลซ์ BVP (S, τ, K, r, σ) และ V ตามเงื่อนไข Lower Boundary ($S \rightarrow 0$) และ Upper Boundary ($S \rightarrow S_{max}$) ผ่าน *class DataGenerator* เป็นจำนวน n ชุด (n_sample_data) บนช่วงตามตาราง 1 โดยค่า K จะถูกสุ่มแบบไม่ต่อเนื่อง ($K_step: 1000.0$) ขณะที่ S จะมีการสุ่มแบบ Uniform รอบค่า K โดยมีค่าต่ำสุดคือ 0 และ

ค่าสูงสุดคือสองเท่าของ K (อิงตาม Moneyness Range) ส่วน r และ σ ก็จะสุ่มตามช่วงของตัวเองตามตาราง 1 หลังจากนั้นให้โมเดลทำนายค่า V โดยใช้ข้อมูลสมนอร์มัลไลซ์ BVP (S, τ, K, r, σ) พร้อมกับคำนวณค่า $\mathcal{L}_{BVP}$ ผ่าน $nn.MSELoss()$ เทียบกับ V ตามเงื่อนไข BVP

3.1.4 สร้างข้อมูลสมนอร์มัลไลซ์ Kink (S, τ, K, r, σ) เป็นจำนวน $n \times 0.5$ โดยพิจารณาที่ $\tau = 0$ และ $S = K$ ส่วน τ, r และ σ ใช้ข้อมูลจาก IVP ในข้อ 3.1.2 สร้างผลเฉลยเป็น $V = 0$ หลังจากนั้นให้โมเดลทำนายค่า V โดยใช้ข้อมูลสมนอร์มัลไลซ์ Kink (S, τ, K, r, σ) พร้อมกับคำนวณค่า $\mathcal{L}_{Kink}$ ผ่าน $nn.MSELoss()$ เทียบกับ $V = 0$

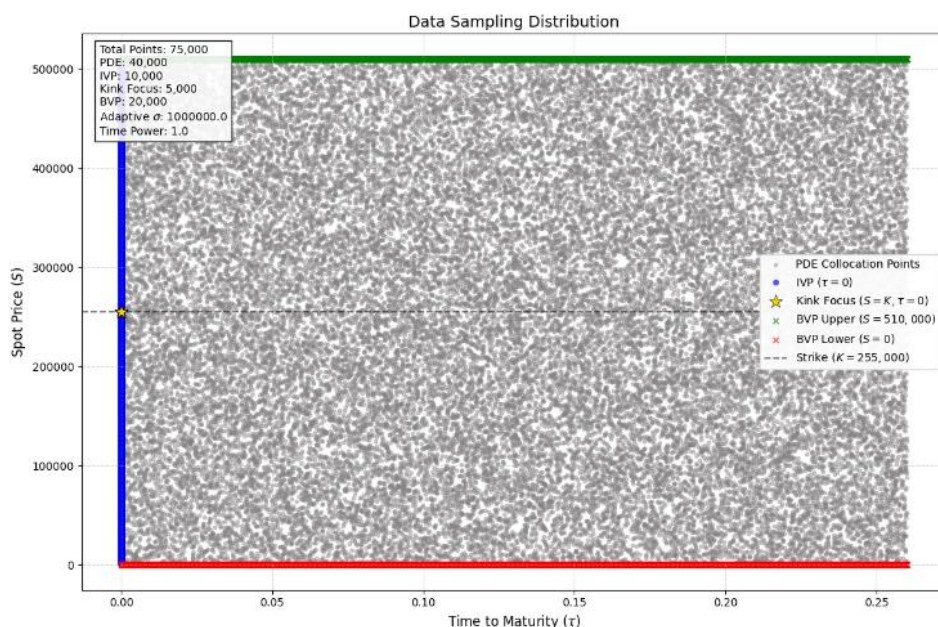
3.1.5 สร้างข้อมูลสมนอร์มัลไลซ์ PDE (S, τ, K, r, σ) ผ่าน *class DataGenerator* เป็นจำนวน $n \times 4$ ชุด โดยค่า K จะถูกสุ่มแบบไม่ต่อเนื่อง (K_step: 1000.0) ขณะที่ S จะมีการสุ่มแบบ Uniform รอบค่า K โดยมีค่าต่ำสุดคือ 0 และค่าสูงสุดคือสองเท่าของ K (อิงตาม Moneyness Range) ส่วน τ, r และ σ ก็จะสุ่มตามช่วงของตัวเองตามตาราง 1 หลังจากนั้นให้โมเดลทำนายค่า V โดยใช้ข้อมูลสมนอร์มัลไลซ์ PDE (S, τ, K, r, σ) พร้อมกับคำนวณค่า $\mathcal{L}_{PDE}$ ผ่าน $nn.MSELoss()$ เทียบกับสมการ BSPDE สมการ (3)

3.1.6 คำนวณ $\mathcal{L}_{total}$ ตามสมการ (4) และทำการปรับแต่งค่าพารามิเตอร์ (Optimize) ผ่านการแพร่ย้อนกลับ (Backpropagation)

3.1.7 เทรนโมเดลจนครบ (Total Epochs) จากนั้นจะได้ไฟล์ model.pth

4. ผลการทดลองและวิเคราะห์ข้อมูล

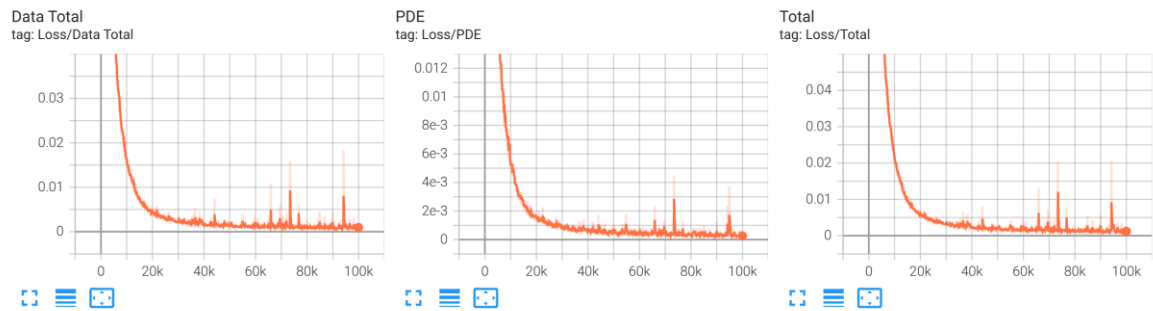
ก่อนฝึกโมเดลระบบจะมีการบันทึกภาพตัวอย่างการกระจายตัวของชุดข้อมูลของทั้งค่าขอบและข้อมูลสำหรับ PDE ดังภาพ 2 ซึ่งจะเป็นถูกใช้ฝึกโมเดลเป็นจำนวนมาก ระหว่างการฝึกโมเดลระบบจะบันทึกค่า Loss และ Validation ทุกๆ 10 รอบการฝึกสู่ TensorBoard เพื่อเฝ้าติดตามความคืบหน้าการเรียนรู้ของโมเดล และเพื่อความปลอดภัยในการเทรนเซิร์ฟเวอร์คอมพิวเตอร์ชุดของระบบจะบันทึกไฟล์โมเดลออกมาทุกๆ 10,000 รอบการเทรน หลังการฝึกโมเดลเสร็จสิ้นระบบจะบันทึกภาพ 4 และ 5 ออกมาเพื่อตรวจสอบคุณภาพของโมเดล



ภาพ 2 ตัวอย่างการกระจายตัวของชุดข้อมูล (Data Sampling) แบบ Uniform Distribution ที่ใช้ในการฝึก PINN Model

ที่มา. Matplotlib จากการทดลอง

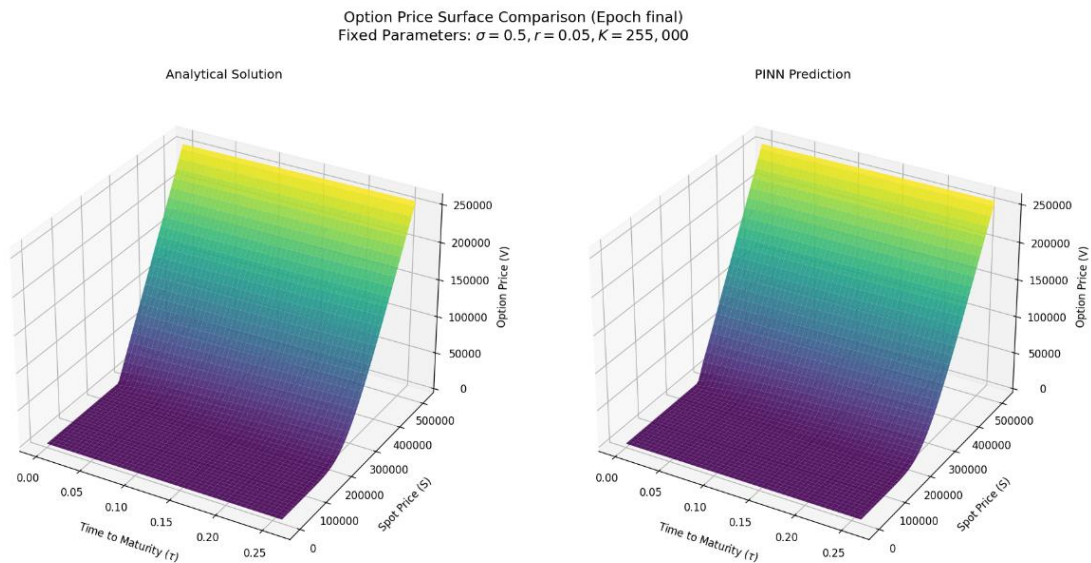
Loss



ภาพ 3 ค่าฟังก์ชันความสูญเสีย (Loss) ของ PINN Model ที่ฝึก 100,000 รอบ

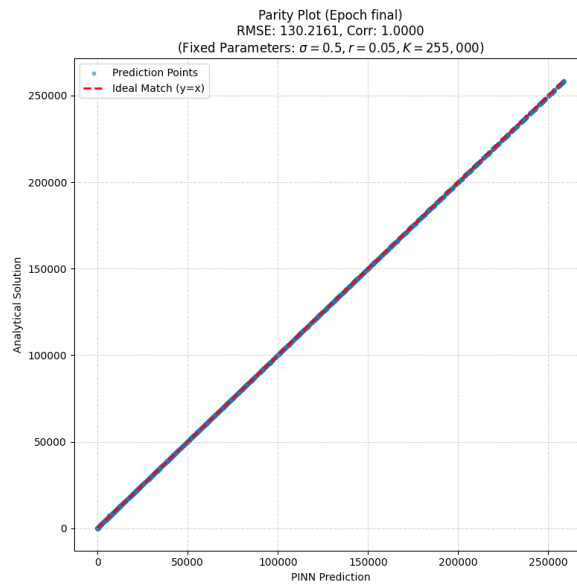
ที่มา. *TensorBoard* จากการทดลอง

จากภาพ 2 แสดงให้เห็นการกระจายตัวของชุดข้อมูลบนมิติ S และ τ ของ 1 รอบการฝึก พบว่ามีจำนวนข้อมูล (S, τ, K, r, σ) มากถึง 75,000 ชุด ซึ่งแต่ละรอบการฝึกการสุ่มจะมีความเป็นอิสระต่อกัน (Unique Scenarios per Epoch) จึงกล่าวได้ว่าโมเดลถูกสอนด้วยข้อมูลในรูปแบบต่างๆ แบบไม่ซ้ำและได้เห็นสถานการณ์ของตลาดอุปขึ้นมามากกว่า 7,500,000,000 สถานการณ์ (Scenarios) (7.5 พันล้านชุดข้อมูล) ซึ่งทำให้โมเดลมีการเรียนรู้รูปแบบครอบคลุมที่สุด (Generalization) ขณะที่ภาพ 3 นั้นแสดงให้เห็นว่าโมเดลเรียนรู้ได้อย่างรวดเร็วและราบเรียบ โดยมีค่า $\mathcal{L}_{Data} = 0.001303411555$ $\mathcal{L}_{PDE} = 0.000172622924$ และ $\mathcal{L}_{total} = 0.001476034522$



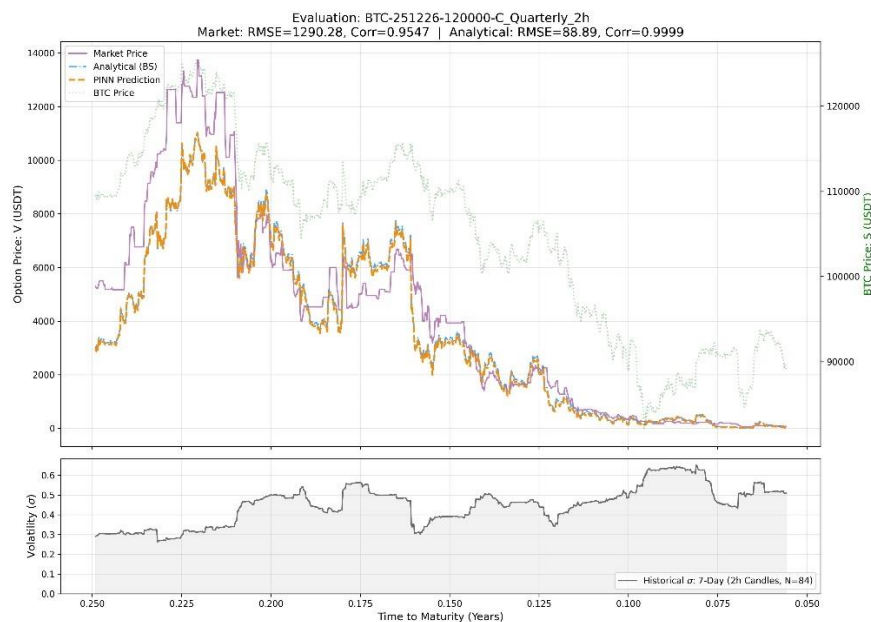
ภาพ 4 การเปรียบเทียบพื้นผิว 3 มิติของของราคาอปชั่น (Call) ที่ได้จากผลเฉลยเชิงวิเคราะห์ (Analytical Solution) และ PINN Model

ที่มา. *Matplotlib* จากการทดลอง



ภาพ 5 เปรียบเทียบผลเฉลยเชิงวิเคราะห์ และ ราคาประมาณจาก PINN Model เมื่อเส้นประสีแดงคือเส้นเปรียบเทียบในอุดมคติ (*Ideal Match*) $y = x$

ที่มา. *Matplotlib* จากการทดลอง



ภาพ 6 ตัวอย่างข้อมูลการเปรียบเทียบระหว่างราคาอปชันจริง (เส้นทึบสีม่วง) ราคาอปชันจากผลเฉลยเชิงวิเคราะห์ (เส้นประสีฟ้า) และราคาอปชันจาก PINN Model (เส้นประสีส้ม) บนเส้นทางราคามิตคอยน์จริง (เส้นประสีเขียว) บนสัญญา BTC-251226-120000-C ประเภทวันหมดอายุ 3 เดือน ประเมินบนกรอบเวลา 2 ชั่วโมงและค่าความผันผวน (σ) ย้อนหลัง 7 วัน โดยมี RMSE = 88.89 และ Corr = 0.9999

ที่มา. *Matplotlib* จาก *scripts/eval_real_market.py*

โมเดลที่ได้รับการฝึกได้เรียนรู้ข้อมูลได้ถึง 5 มิติ ทำให้เป็นเรื่องยากที่จะแสดงกายภาพของมิติที่สูงกว่าได้ ดังนั้นการวิจัยครั้งนี้จึงเลือกที่จะแสดงผลพื้นผิวผลเฉลยผ่านข้อมูล 2 มิติของ S และ τ แทน โดยสร้างเป็น Meshgrid ที่มีการกระจายข้อมูลเท่าๆกัน (ผ่าน *np.linspace*)

ขนาด 100×100 ส่วน $K = 255000, r = 0.05$ และ $\sigma = 0.5$ (ถูกปรับให้เป็นค่าคงที่) ดังที่แสดงในภาพ 4 ผลคือพื้นผิวผลเฉลยของโมเดลที่ทำนายออกมานั้นสอดคล้องเป็นอย่างดีกับพื้นผิวผลเฉลยจากผลเฉลยเชิงวิเคราะห์ และเพื่อวัดค่าประสิทธิภาพได้จึงต้องวัดค่า RMSE (Root Mean Squared Error) เพื่อบ่งบอกว่าโดยเฉลี่ยแล้วราคาจากโมเดลทำนายห่างจากราคาจากผลเฉลยเชิงวิเคราะห์แค่ไหน (ยิ่งค่าน้อยยิ่งดี) และวัดค่า Corr (Correlation) เพื่อดูว่าขนาดและทิศทางของค่าที่ทำนายได้กับค่าผลเฉลยจริงนั้นสอดคล้องกันไหม (ยิ่งค่าเข้าใกล้ 1 ยิ่งดี) ผลลัพธ์จึงออกมาดังภาพ 5 ซึ่งวัดค่า RMSE = 130.2161 และ Corr = 1.0 ซึ่งบ่งบอกว่าโมเดลเข้าใจฟิสิกส์ได้เป็นอย่างดี เมื่อลองนำโมเดลไปใช้ทำนายกับตลาดออปชันจริงบนการวิ่งของสินทรัพย์อย่างอิงจริงในช่วงเวลานั้น (บนสัญญา BTC-251226-120000-C) ดังที่แสดงในภาพ 6 สามารถวัดค่า RMSE = 88.89 และ Corr = 0.9999 ซึ่งมีความแม่นยำในระดับใช้งานได้เลยทีเดียว จุดเด่นสำคัญอย่างหนึ่งของโมเดลคือสามารถใช้ Automatic Differentiation (AD) (แสดงในภาพ 1) คำนวณค่าเชิงอนุพันธ์ (เช่นค่า *Greeks*) ได้อย่างแม่นยำผ่านกระบวนการ Backpropagation เมื่อทดสอบโมเดล API ผ่าน Swagger UI ที่รันบน Docker Container แล้วลองส่งค่าพบว่าโมเดลทำนายราคาออปชันพร้อมค่า Greeks ออกมาได้อย่างรวดเร็วในระดับ 1.1478 มิลลิวินาทีดังในภาพ 7 และวัดประสิทธิภาพออกมาเทียบกับค่าผลเฉลยจริงได้ดังตาราง 2 ซึ่งแม่นยำในระดับผลต่างเพียงราวๆ 1%

Curl	Response body
<pre>curl -X 'POST' \ 'http://localhost:8000/v1/predict' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "option_type": "call", "request_id": "exp-2026-batch1", "risk_free_rate": 0.05, "spot_price": 96000, "strike_price": 95000, "time_to_maturity": 0.15, "volatility": 0.5 }'</pre>	<pre>{ "price": 8143.612593412399, "greeks": { "delta": 0.5742797636985778, "gamma": 0.00002130480239868164, "theta": -26789.25458055276, "vega": 14596.712589263916, "rho": 7221.659334997336 }, "model_version": "call", "inference_time_ms": 1.1478, "device": "cpu" }</pre>

ภาพ 7 ตัวอย่างการทดสอบโมเดล API ผ่าน Swagger UI ที่รันบน Docker Containerพบว่าโมเดลสามารถทำนายค่าออกมาได้เร็วในระดับมิลลิวินาที

ที่มา. *Swagger UI* จากการทดลอง

ตาราง 2

ความแม่นยำของข้อมูลที่ได้จาก PINN API ในภาพ 7

ค่าที่วัด	ค่าจากทฤษฎี	ค่าจากโมเดล	ผลต่าง	ผลต่างเปอร์เซ็นต์
Price (Call)	8,225.35	8,143.61	-81.74	-0.99%
Delta (Δ)	0.5752	0.5743	-0.0009	-0.16%
Gamma (Γ)	0.00002108	0.0000213	+0.00000022	+1.04%
Theta (Θ)	-26,630.73	-26,789.25	-158.52	+0.60%
Vega (V)	14,568.63	14,596.71	+28.08	+0.19%
Rho (ρ)	7,049.07	7,221.66	+172.59	+2.45%

ที่มา. คำนวณและประมวลผลโดยใช้ Google Gemini (LLM-based Computation)

5. สรุปผลการวิจัยและข้อเสนอแนะ

จากการทดลองพัฒนาและทดสอบประสิทธิภาพของโมเดล PINN พบว่าเอไอสามารถเข้าใจแบบจำลองแบล็ก-โชลส์ ได้เป็นอย่างดีโดยแสดงผลพื้นผิวผลเฉลยได้ใกล้เคียงกันมากๆ โดยมีค่า RMSE = 130.2161 และ Corr = 1.0 ขณะที่เมื่อนำไปใช้งานจริงกับตลาดจริงสามารถวัดค่า RMSE = 88.89 และ Corr = 0.9999 การเรียนรู้บนช่วงข้อมูลที่มีหลายมิติ (S, τ, K, r, σ) ทำให้โมเดลเห็นภาพรวมของตลาดอนุพันธ์ได้และอาจเรียนรู้ได้ดียิ่งขึ้นหากมีการปรับปรังค่าพารามิเตอร์ต่างๆ (ในไฟล์ `configs/train_config.yaml`) รวมถึงการลองปรับแต่งโครงข่ายประสาทเทียมในรูปแบบอื่นๆ โดยคอมพิวเตอร์ที่มีความแรงสูงจะได้ประโยชน์อย่างยิ่งต่อการพัฒนาโมเดลให้มีประสิทธิภาพสูง อย่างไรก็ตามการวิจัยนี้ก็ได้อาศัยให้ดูว่าการฝึกเอไอแบบ PINN สามารถทำให้เอไอเข้าใจกฎฟิสิกส์ต่างๆได้ ซึ่งรวมถึงการแก้ปัญหาย้อนกลับเพื่อค้นหาพารามิเตอร์ที่ไม่ทราบค่าได้อย่างเช่นค่า r และ σ จากข้อมูลราคาอปชันจริง หรือแม้แต่การค้นหาค่าความเสี่ยง (Greeks) เช่น Delta (Δ), Gamma (Γ), Vega (v), Theta (θ), และ Rho (ρ) ได้ทันทีผ่านกระบวนการ AD ในระดับ 1.1478 มิลลิวินาที ดังภาพ 7 ประโยชน์ที่นักวิเคราะห์การลงทุนจะได้รับก็คือการทราบราคาอปชันยุติธรรมและค่า Greeks แบบทันทีทันใดทำให้สามารถบริหารจัดการความเสี่ยงและพอร์ตการลงทุนได้แบบเวลาจริง (Real-time) โมเดล PINN จึงเป็นเครื่องมือที่ดีสำหรับใช้วิเคราะห์จุดเข้าออกที่ได้เปรียบที่สุดในตลาดอนุพันธ์ที่มีความผันผวนสูง

แม้ในงานวิจัยนี้จะเลือกทดสอบกับสมการ Black-Scholes (European) ซึ่งเป็นแบบจำลองมาตรฐาน แต่ศักยภาพที่แท้จริงของ PINN คือความสามารถในการแก้ปัญหาแบบจำลองทางการเงินที่ซับซ้อนและปราศจากผลเฉลยแน่นอนตรง (Analytical Solution) ซึ่งเป็นข้อจำกัดที่วิธีการคำนวณเชิงตัวเลขแบบดั้งเดิม (Numerical) ทำได้ยาก เช่น ออปชันที่อ้างอิงกับสินทรัพย์จำนวนมาก (High-dimensional Basket Options) American Options หรือแบบจำลองความผันผวนแบบสุ่ม (Stochastic Volatility Models) อื่นๆ เช่น Heston Model

6. ภาคผนวก

ชุดคำสั่งที่พัฒนาขึ้นเพื่อใช้วิจัยในงานวิจัยนี้

[1] https://github.com/pornchai1889/pinn_btcsdt_option

7. บรรณานุกรม

- [1] Black, F., & Scholes, M. (1973). *The Pricing of Options and Corporate Liabilities. Journal of Political Economy*, 81(3), 637-654.
- [2] Dhiman, A., & Hu, Y. (2023). *Physics Informed Neural Network for Option Pricing*. arXiv preprint arXiv:2312.06711.
- [3] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). *Physics-informed Neural networks: A deep learning framework for solving forward and Inverse problems involving nonlinear partial differential equations. Journal of Computational Physics*, 378, 686-707.